

Module 3

UVM Basics

Learning objectives

- Master UVM class hierarchy and phases

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Introduction to UVM"]

T2["UVM Class Hierarchy"]

T1 --> T2

T3["UVM Phases"]

T2 --> T3

Master UVM class hierarchy and phases

Overview

- This module introduces the Universal Verification Methodology (UVM) and its implementation in pyuvm...

How to learn this module

Suggested learning path

- Solid cocotb skills from Module 2 — you will reuse trigger timing inside pyuvvm drivers later
- Read ``module3/dut/simple_blocks/adder.v`` — combinational operands ``a``, ``b``, output ``sum``
- Step through ``module3/examples/phases/`` and ``class_hierarchy/`` before integrated ``test_simple_uvm.p...`
- Understand UVM phase order: build → connect → run (with objections) → report
- ConfigDB and factory examples are isolated; run them before merging concepts in the pyuvvm test

Design architecture

1. RTL and interfaces

- Primary DUT: ``module3/dut/simple_blocks/adder.v`` — combinational operands and sum
- Clock/reset scaffolding for pyuvvm tests that mirror industry TB hooks
- Flat block; no protocol channels — focus stays on UVM structure, not bus complexity

Rtl Architecture

Diagram: Rtl Architecture
(render with mmdc when available)
flowchart LR
A[operands a,b] --> ADD[adder.v]
ADD --> SUM[sum]
CLK[clk / rst] --> ADD

2. UVM testbench architecture

- Hierarchy: ``uvm_test`` →
 ``uvm_env`` → ``uvm_agent`` (driver,
 monitor, sequencer) → DUT
- ``uvm_sequence_item`` /
 ``uvm_sequence`` carry
 transactions; driver converts
 items to pin wiggles
- ConfigDB publishes virtual
 interfaces and parameters before
 ``build_phase``
- Factory + objections control
 creation and phase lifetime

Verification Architecture

Diagram: Verification Architecture
(render with mmdc when available)
flowchart TB
T[uvm_test] --> E[uvm_env]
E --> AG[uvm_agent]
AG --> SEQ[sequencer / sequences]
AG --> DRV[driver]
AG --> MON[monitor]

3. Execution pipeline

- Build: Verilator compiles adder RTL; pyuvmm
 `build\_phase` constructs test/env/agent/component tree
- Connect: TLM ports linked in `connect\_phase`;
 ConfigDB delivers virtual interface handles to driver...
- Sim/run: sequences start on sequencer; driver
 applies operands; objections hold run phase until seq...
- Check/report: monitor samples `sum`; UVM report
 phase summarizes errors; expect UVM TEST PASSED

4. Module layout

- ``module3/examples/`` — isolated API demos (phases, reporting, configdb, factory, objections)
- ``module3/tests/pyuvm_tests/`` — integrated ``make SIM=verilator TEST=test_simple_uvm`` flows
- Each example directory has its own Makefile for incremental learning
- Assessment maps each example topic to integrated test behaviors

DUT — 8-bit adder

```
module adder (  
    input  wire      clk,  
    input  wire      rst_n,  
    input  wire [7:0] a,  
    input  wire [7:0] b,  
    output reg [7:0] sum,  
    output reg       carry  
);  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin  
            sum <= 8'h00;  
            carry <= 1'b0;  
        end else begin  
            {carry, sum} <= a + b;  
        end  
    end
```

...

UVM transaction item

```
class AdderTransaction(uvm_sequence_item):
    """Transaction for adder test."""

    def __init__(self, name="AdderTransaction"):
        super().__init__(name)
        self.a = 0
        self.b = 0
        self.expected_sum = 0
        self.expected_carry = 0

    def __str__(self):
        return (f"a=0x{self.a:02X}, b=0x{self.b:02X}, "
                f"expected_sum=0x{self.expected_sum:02X}, "
                f"expected_carry={self.expected_carry}")
```

Key files to study

Open these in the repo

- ``module3/dut/simple_blocks/adder.v`` — primary combinational DUT for UVM structure labs
- ``module3/examples/phases/phases_example.py`` — phase callbacks and simulation lifetime
- ``module3/examples/configdb/configdb_example.py`` — virtual interface and parameter publishing
- ``module3/examples/factory/factory_example.py`` — type overrides without hierarchy edits
- ``module3/tests/pyuvm_tests/test_simple_uvm.py`` — integrated test → env → agent → adder DUT
- ``scripts/module3.sh`` — per-topic flags (``--phases``, ``--factory``, ...) plus ``--pyuvm-tests``

Verification & testing methods

1. Phase-based test execution

- build → connect →
end_of_elaboration →
start_of_simulation → run →
extract → report
- Objections in run phase prevent premature finish until sequences complete
- Reporting verbosity filters
debug vs error messages

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

B[build_phase] --> C[connect_phase]

C --> R[run_phase + objections]

R --> X[extract / report]

X --> P{UVM TEST PASSED?}

2. Stimulus and checking

- Directed sequences on adder operands; monitor samples outputs for scoreboard hooks
- Factory overrides swap test/agent types without editing hierarchy code
- ConfigDB sets agent active/passive and interface handles

3. Step-by-step lab execution

- 1. API drills: ``./scripts/module3.sh --class-hierarchy --phases --configdb``
- 2. Factory/objections: ``./scripts/module3.sh --factory --objections``
- 3. Integrated test: ``./scripts/module3.sh --pyuvm-tests`` (or ``make TEST=test_simple_uvm``)
- 4. Confirm simulation log shows phase progression and no UVM_ERROR/FATAL
- 5. Optional: factory override exercise from examples, re-run integrated test to observe swapped typ...

4. Closure

- ``./scripts/module3.sh --pyuvm-tests`` and per-topic flags (``--phases``, ``--factory``, ...)
- Pass when simulation report shows UVM TEST PASSED and no fatal errors
- Assessment: class hierarchy, phases, configdb, factory, objections

Syllabus topics

1. Introduction to UVM (1/3)

- What is UVM?
- Universal Verification Methodology
- Industry standard for verification
- Methodology vs library
- History and evolution
- Why UVM?

1. Introduction to UVM (2/3)

- Reusability
- Standardization
- Scalability
- Maintainability
- UVM in Python (pyuvm)
- Python implementation of UVM 1.2

1. Introduction to UVM (3/3)

- Advantages over SystemVerilog UVM
- Compatibility and features

2. UVM Class Hierarchy (1/4)

- Base Classes
 - ``uvm_object`` - Base for all UVM objects
 - ``uvm_component`` - Base for all UVM components
- Differences and use cases
- Component Classes
 - ``uvm_test`` - Top-level test class

2. UVM Class Hierarchy (2/4)

- ``uvm_env`` - Environment container
- ``uvm_agent`` - Agent (driver, monitor, sequencer)
- ``uvm_driver`` - Drives transactions to DUT
- ``uvm_monitor`` - Monitors DUT signals
- ``uvm_sequencer`` - Manages sequences
- ``uvm_scoreboard`` - Checks results

2. UVM Class Hierarchy (3/4)

- Object Classes
 - ``uvm_sequence_item`` - Transaction objects
 - ``uvm_sequence`` - Sequence of transactions
 - ``uvm_config_object`` - Configuration objects
- Class Relationships
- Inheritance hierarchy

2. UVM Class Hierarchy (4/4)

- Composition patterns
- Factory pattern

3. UVM Phases (1/6)

- Phase Overview
- Why phases exist
- Phase execution order
- Phase synchronization
- Phase types
- Build Phases

3. UVM Phases (2/6)

- ``build_phase()` - Component construction
- ``connect_phase()` - Component connections
- ``end_of_elaboration_phase()` - Final setup
- Run Phases
- ``run_phase()` - Main test execution
- ``pre_reset_phase()` - Before reset

3. UVM Phases (3/6)

- ``reset_phase()` - Reset sequence
- ``post_reset_phase()` - After reset
- ``pre_configure_phase()` - Before configuration
- ``configure_phase()` - Configuration
- ``post_configure_phase()` - After configuration
- ``pre_main_phase()` - Before main test

3. UVM Phases (4/6)

- ``main_phase()` - Main test execution
- ``post_main_phase()` - After main test
- ``pre_shutdown_phase()` - Before shutdown
- ``shutdown_phase()` - Shutdown sequence
- ``post_shutdown_phase()` - After shutdown
- Cleanup Phases

3. UVM Phases (5/6)

- ``extract_phase()` - Extract results
- ``check_phase()` - Final checks
- ``report_phase()` - Generate reports
- ``final_phase()` - Final cleanup
- Phase Implementation
- Synchronous phases (build, connect)

3. UVM Phases (6/6)

- Asynchronous phases (run phases)
- Phase methods
- Phase ordering

4. UVM Reporting System (1/5)

- Reporting Overview
- UVM messaging system
- Severity levels
- Verbosity levels
- Message formatting
- Severity Levels

4. UVM Reporting System (2/5)

- `UVM\_FATAL` - Fatal errors
- `UVM\_ERROR` - Errors
- `UVM\_WARNING` - Warnings
- `UVM\_INFO` - Informational
- `UVM\_DEBUG` - Debug messages
- Verbosity Levels

4. UVM Reporting System (3/5)

- `UVM\_NONE` - No messages
- `UVM\_LOW` - Low verbosity
- `UVM\_MEDIUM` - Medium verbosity
- `UVM\_HIGH` - High verbosity
- `UVM\_FULL` - Full verbosity
- `UVM\_DEBUG` - Debug verbosity

4. UVM Reporting System (4/5)

- Using Reporting
- ``self.logger.info()``
- ``self.logger.warning()``
- ``self.logger.error()``
- ``self.logger.fatal()``
- Message formatting

4. UVM Reporting System (5/5)

- Verbosity control

5. UVM Configuration Database (ConfigDB) (1/4)

- ConfigDB Overview
- What is ConfigDB?
- Why use ConfigDB?
- Configuration hierarchy
- Setting Configuration
- ``ConfigDB().set()``

5. UVM Configuration Database (ConfigDB) (2/4)

- Configuration paths
- Configuration objects
- Scalar configuration
- Getting Configuration
- ``ConfigDB().get()``
- Configuration lookup

5. UVM Configuration Database (ConfigDB) (3/4)

- Default values
- Configuration checking
- Configuration Patterns
- Agent configuration
- Environment configuration
- Test configuration

5. UVM Configuration Database (ConfigDB) (4/4)

- Hierarchical configuration

6. Factory Pattern (1/2)

- Factory Overview
- What is the factory?
- Why use factory?
- Factory benefits
- Factory Usage
- Type registration

6. Factory Pattern (2/2)

- Object creation
- Override mechanism
- Factory patterns

7. First UVM Test Class (1/3)

- Test Structure
- Test class definition
- Inheriting from `uvm\_test`
- Required methods
- Test organization
- Environment Creation

7. First UVM Test Class (2/3)

- Creating environment
- Environment hierarchy
- Component instantiation
- Test Execution
- ``run_phase()` implementation
- Objection mechanism

7. First UVM Test Class (3/3)

- Test flow
- Completion

8. Environment Structure (1/3)

- Environment Basics
- What is environment?
- Environment purpose
- Environment structure
- Environment Components
- Agent instantiation

8. Environment Structure (2/3)

- Scoreboard instantiation
- Coverage instantiation
- Other components
- Environment Connections
- Component connections
- Analysis port connections

8. Environment Structure (3/3)

- TLM connections

9. Objection Mechanism (1/3)

- Objections Overview
- What are objections?
- Why objections?
- Objection lifecycle
- Using Objections
- ``raise_objection()``

9. Objection Mechanism (2/3)

- ``drop_objection()``
- Objection timing
- Multiple objections
- Objection Patterns
- Test objections
- Sequence objections

9. Objection Mechanism (3/3)

- Component objections
- Best practices

10. UVM Test Execution (1/3)

- Test Flow
- Test startup
- Phase execution
- Test completion
- Cleanup
- Running Tests

10. UVM Test Execution (2/3)

- ``uvm_root().run_test()``
- Test selection
- Test parameters
- Test execution
- Test Organization
- Multiple tests

10. UVM Test Execution (3/3)

- Test inheritance
- Test libraries
- Test selection

Command reference highlights

Isolated UVM API examples

- ``./scripts/module3.sh --phases --reporting`` — phase and verbosity drills
- ``./scripts/module3.sh --configdb --factory --objections`` — ConfigDB, factory, objection tracks
- ``./scripts/module3.sh --class-hierarchy`` — component tree and TLM port basics

Integrated pyuvvm simulation

- ``./scripts/module3.sh --pyuvvm-tests`` — default integrated regression against adder DUT
- ``cd module3/tests/pyuvvm_tests && make SIM=verilator TEST=test_simple_uvm`` — direct Make debug path
- ``./scripts/module3.sh --skip-examples --pyuvvm-tests`` — jump to integrated test when API drills are...

Verbosity and overrides

- Set ``PYUVM_VERBOSITY`` or `plusargs` (when supported) to filter UVM report noise during debug
- Factory overrides in examples swap test/agent types
 - mirror pattern in integrated test
- Re-run single example Make target under ``module3/examples/<topic>/'` for fast iteration

Hands-on examples

Module 3 orchestrator

```
$ ./scripts/module3.sh --help
```

Terminal: module3_check

```
[[0;36m=====0m
[[0;36mModule 3: UVM Basics0m
[[0;36m=====0m
```

Usage: ./scripts/module3.sh [OPTIONS]

Module 3: UVM Basics

This script runs examples and tests for Module 3.

OPTIONS:

Examples:

--class-hierarchy	Run UVM class hierarchy examples
--phases	Run UVM phases examples
--reporting	Run UVM reporting examples
--configdb	Run ConfigDB examples
--factory	Run factory pattern examples
--objections	Run objection mechanism examples
--all-examples	Run all examples (default)
--skip-examples	Skip all examples

Tests:

--pyuvvm-tests	Run pyuvvm tests
----------------	------------------

Environment:

--venv DIR	Virtual environment directory (default: .venv)
--no-venv	Don't use virtual environment
--sim SIMULATOR	Simulator to use (default: verilator)

Other:

--help, -h	Show this help message
------------	------------------------

EXAMPLES:

```
# Run all examples
./scripts/module3.sh
```


Exercise scaffold

```
./scripts/module3.sh --scaffold
```

Demo: Class Hierarchy

```
$ .venv/bin/python  
    module3/examples/class_hierarchy/class_hierarchy_example.py
```

Terminal: ex01_class_hierarchy

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module3/examples/class_hi
import cocotb

ModuleNotFoundError: No module named 'cocotb'

Demo: UVM Phases

```
$ .venv/bin/python module3/examples/phases/phases_example.py
```

Terminal: ex02_phases

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module3/examples/phases/p
import cocotb

ModuleNotFoundError: No module named 'cocotb'

Demo: UVM Reporting

```
$ .venv/bin/python module3/examples/reporting/reporting_example.py
```

Terminal: ex03_reporting

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module3/examples/reportin

import cocotb

ModuleNotFoundError: No module named 'cocotb'

Demo: ConfigDB

```
$ .venv/bin/python module3/examples/configdb/configdb_example.py
```

Terminal: ex04_configdb

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module3/examples/configdb
import cocotb

ModuleNotFoundError: No module named 'cocotb'

Demo: Factory Pattern

```
$ .venv/bin/python module3/examples/factory/factory_example.py
```

Terminal: ex05_factory

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module3/examples/factory/
import cocotb

ModuleNotFoundError: No module named 'cocotb'

Demo: Objection Mechanism

```
$ .venv/bin/python module3/examples/objections/objections_example.py
```

Terminal: ex06_objections

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module3/examples/objectio

import cocotb

ModuleNotFoundError: No module named 'cocotb'

Practice & assessment

What you should know (1/3)

- Understand UVM methodology
- Explain UVM class hierarchy
- Understand and use UVM phases
- Use UVM reporting effectively
- Use ConfigDB for configuration
- Understand factory pattern

What you should know (2/3)

- Create UVM test classes
- Structure UVM environments
- Use objection mechanism
- Execute UVM tests
- Test class definition
- Basic phases

What you should know (3/3)

- Objection mechanism

Exercises

- Test Class Creation
- Environment Structure
- Reporting
- Configuration
- Phase Understanding

Assessment checklist

- Understands UVM methodology
- Can explain class hierarchy
- Understands all UVM phases
- Can use UVM reporting
- Can use ConfigDB
- Understands factory pattern

Summary & next steps

- Master UVM class hierarchy and phases
- Complete module3/CHECKLIST.md
- Review module3/EXAMPLES.md and run each lab
- Next: Next module in course